

Project_3_SQL

June 17, 2026

0.1 PROJECT 3 SQL DATA ANALYSIS

```
[1]: import pandas as pd
import sqlite3

# 1. Ingest your high-integrity clean CSV dataset
csv_data = "Clean_Dataset_Data_Analytics.csv"
df = pd.read_csv(csv_data)

# 2. Establish a connection to an in-memory relational SQLite database
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

# 3. Write the clean DataFrame directly into a SQL table named ↵
    ↵ 'sales_transactions'
df.to_sql("sales_transactions", conn, index=False, if_exists="replace")

print(" Success: Relational database table 'sales_transactions' is live!")
print(" Data Schema and 1,200 records are fully indexed for SQL query ↵
    ↵ processing.")
```

Success: Relational database table 'sales_transactions' is live!
Data Schema and 1,200 records are fully indexed for SQL query processing.

0.1.1 Strategic Filter & Sorting (SELECT, WHERE, ORDER BY)

Extract all high-value transactions from top marketing acquisition channel (Instagram) where customers bought multiple high-ticket items, sorting from largest sale to smallest:

```
[2]: query_1 = """
SELECT OrderID, Date, CustomerID, Product, Quantity, TotalPrice, ReferralSource
FROM sales_transactions
WHERE ReferralSource = 'Instagram' AND Quantity >= 4
ORDER BY TotalPrice DESC;
"""
pd.read_sql_query(query_1, conn)
```

```
[2]:      OrderID      Date CustomerID Product  Quantity  TotalPrice  \
0   ORD200107  2023-03-27    C16775  Printer         5      3353.75
```

1	ORD200463	2023-05-26	C25276	Laptop	5	3313.90
2	ORD200367	2024-04-25	C13108	Laptop	5	3293.85
3	ORD200010	2023-12-29	C43443	Tablet	5	3129.85
4	ORD200450	2024-01-15	C36408	Monitor	5	3075.50
..	...	...	...	...	...	...
96	ORD201100	2023-08-17	C15146	Laptop	5	190.55
97	ORD200231	2023-11-05	C30955	Phone	4	167.60
98	ORD200834	2024-04-11	C98483	Printer	5	110.70
99	ORD200677	2025-04-03	C56722	Tablet	5	110.35
100	ORD200602	2024-03-05	C35852	Tablet	4	104.16

	ReferralSource
0	Instagram
1	Instagram
2	Instagram
3	Instagram
4	Instagram
..	...
96	Instagram
97	Instagram
98	Instagram
99	Instagram
100	Instagram

[101 rows x 7 columns]

Analytical Insight:

The extraction isolated a reliable core of **101** high-volume consumer baskets purely driven by social media acquisition (**Instagram**). Wholesale and multi-item ordering behavior is heavily centered around **laptop** and **printer** inventory blocks within this channel.

0.1.2 Metrics Aggregation & Volume Bucketing (GROUP BY, SUM, AVG, COUNT)

Calculate the exact metrics needed to assess how different payment types perform by volume, total value, and average basket size:

```
[3]: query_2 = """
SELECT
    PaymentMethod,
    COUNT(OrderID) AS Total_Transactions,
    SUM(Quantity) AS Total_Units_Sold,
    ROUND(SUM(TotalPrice), 2) AS Gross_Revenue,
    ROUND(AVG(TotalPrice), 2) AS Average_Order_Value
FROM sales_transactions
GROUP BY PaymentMethod
ORDER BY Gross_Revenue DESC;
"""
```

```
pd.read_sql_query(query_2, conn)
```

```
[3]: PaymentMethod  Total_Transactions  Total_Units_Sold  Gross_Revenue  \
0    Credit Card           234             712      263847.63
1         Online           258             731      262442.94
2         Cash            246             753      259786.29
3    Gift Card            230             675      246323.92
4    Debit Card            232             664      232361.18

    Average_Order_Value
0          1127.55
1          1017.22
2          1056.04
3          1070.97
4          1001.56
```

Analytical Insight:

While Online transactions represent the highest order frequency channel (258 entries), **Credit Card** checkouts drive the highest overall top-line financial return (\$263,847.63) and post the largest individual Average Order Value (\$1,127.55).

0.1.3 Advanced Group Filtering (HAVING Clause Constraint)

Pinpoint the specific product inventory categories that have generated more than \$100,000 in gross revenue:

```
[4]: query_3 = """
SELECT
    Product,
    COUNT(OrderID) AS Transaction_Volume,
    SUM(Quantity) AS Total_Quantity_Sold,
    ROUND(SUM(TotalPrice), 2) AS Group_Gross_Revenue
FROM sales_transactions
GROUP BY Product
HAVING Group_Gross_Revenue > 100000.00
ORDER BY Group_Gross_Revenue DESC;
"""
pd.read_sql_query(query_3, conn)
```

```
[4]: Product  Transaction_Volume  Total_Quantity_Sold  Group_Gross_Revenue
0    Chair           178             562      195620.11
1  Printer           181             542      195612.61
2   Laptop           173             535      192126.56
3   Tablet           179             497      186568.95
4  Monitor           163             480      175651.41
5    Desk            170             508      167459.93
6   Phone            156             411      151722.39
```

Analytical Insight:

Exactly 7 core inventory classifications comfortably clear the corporate \$100k valuation filter threshold. **Chairs** remain the primary engine by financial margin performance (\$195,620.11), tightly paired with Printers by transaction scale (181 listings).